

GLARoutes

*A Project Report submitted in partial fulfilment of the
requirements for the award of the degree of*

Master of Computer Applications

By

Manish Singh

2484200112

Bhawana Bhardwaj

2484200043

Nidhi Chaturvedi

2484200127

Group No.: 35

Under the Guidance of

Mr. Shishupal Singh

Department of Computer Engineering & Applications

Institute of Engineering & Technology



GLA University
Mathura -2881406, INDIA
April, 2026



12-B Status from UGC

Department of Computer Engineering and Applications GLA
University, 17 km Stone, NH#2, Mathura-Delhi Road, P.O.
Chaumunhan, Mathura-281406 (U.P.)

Declaration

I hereby declare that the work which is being presented in the MCA Project “**GLARoutes**”, in partial fulfilment of the requirements for the award of the Master of Computer Applications and submitted to the Department of Computer Engineering and Applications of GLA University, Mathura, is an authentic record of my own work carried under the supervision of **Mr. Shishupal Singh** (Technical Trainer).

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign_____

Manish Singh
2484200112

Sign_____

Bhawana Bhardwaj
2484200043

Sign_____

Nidhi Chaturvedi
2484200127

Certificate

This is to certify that the above statements made by the candidate are correct to the best of my/our knowledge and belief.

Supervisor
Mr. Shishupal Singh
(Technical Trainer)
Dept. of Computer Engg. & App

Project Co-ordinator
Mr. Sachendra Singh Chauhan
(Assistant Professor)
Dept. of Computer Engg. & App.

Program Co-ordinator
Mr. Siddharth Singh
(Assistant Professor)
Dept. of Computer Engg. & App.

Date:

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to our Supervisor **Mr. Shishupal Singh**, for his invaluable guidance, mentorship, and constant support throughout the development of our project, **GLARoutes**. His extensive knowledge, patient supervision, and thoughtful feedback were instrumental in overcoming challenges and ensuring the successful completion of this work.

We are sincerely thankful to our Project Coordinator **Dr. Sachendra Singh Chauhan**, for his continuous encouragement and guidance. His valuable suggestions and timely support greatly contributed to the smooth execution of our project.

We would also like to extend our heartfelt thanks to our Program Coordinator **Mr. Siddharth Singh**, for his motivation and support throughout this journey.

Our supervisor's encouragement and insights helped us refine our ideas and provided a clear direction for our efforts. His ability to simplify complex problems and offer practical solutions played a pivotal role in shaping this project. We are particularly grateful for the time and effort he dedicated to reviewing our work and providing constructive suggestions, which significantly enhanced the quality of the final outcome.

This project would not have been possible without his unwavering belief in our abilities and his emphasis on learning and innovation. He has been an inspiring mentor and a constant source of motivation throughout this journey.

Manish Singh 2482200112

Bhawana Bhardwaj 2484200043

Nidhi Chaturvedi 2484200127

ABSTRACT

The rapid expansion of modern educational infrastructure has led to the development of "mega-campuses," where navigating between specialized departmental blocks, administrative zones, and utility centres presents a significant cognitive load for newcomers. GLA Routes is an advanced Android-based geospatial application developed to mitigate these navigational complexities within the GLA University ecosystem. Built using the Java programming language within the Android Studio environment, the application serves as a bridge between static campus architecture and dynamic user mobility.

The core architecture of the system leverages a tripartite integration of the Google Maps SDK, Firebase Realtime Database, and Firebase Authentication. By implementing the Google Maps API, the project provides high-precision, turn-by-turn routing using polylines that connect the user's real-time GPS coordinates to specific localized "nodes" or campus blocks. Unlike generic navigation software, GLA Routes offers a granular "Block-Wise" mapping system, categorized by academic disciplines such as Engineering, Management, and Applied Sciences. To enhance the pre-arrival experience, the system incorporates a 360-degree Panoramic Viewer, allowing users to virtually orient themselves with building facades before physical arrival.

Through a synchronized backend, users can access real-time Placement Analytics, Academic Fee Structures, and Campus Event Calendars. A standout feature of the project is the GLA Free Ride Module.

This report provides a comprehensive walkthrough of the system's requirement specifications, architectural design, database schemas, and rigorous testing phases, concluding with a roadmap for future enhancements such as Augmented Reality (AR) wayfinding and AI-driven campus assistance.

CONTENTS

Declaration	i
Certificate	ii
Acknowledgement	iii
Abstract	iv
List of figures	v
List of Tables	vi
CHAPTER 1 Introduction	1-6
1.1 Overview and Motivation	1
1.2 Objective	1-3
1.2.1 Advanced Geospatial Navigation Development	2
1.2.2 Backend Architecture & Data Integrity	2
1.2.3 Information Accessibility & Student Utility	2
1.2.4 User-Centric Design and Orientation	3
1.2.5 Institutional Impact and Efficiency	3
1.3 Summary of Similar Application	3-6
1.3.1 Google Maps	4
1.3.2 Here WeGo and OpenStreetMap (OSM)	4
1.3.3 Specialized Campus Navigation Systems	4
1.3.4 Comparative Analysis Table	5
1.3.5 The "GLA Routes" USP (Unique Selling Proposition)	5
1.4 Organization of the Project	6
CHAPTER 2 Software Requirement Analysis	7-15
2.1 Function Requirements	7-8
2.2 Non- Functional Requirements	8-9
2.3 Hardware Requirements	10-11
2.4 Software Requirements	11-12
2.4.1 Development Software (IDE & Environment)	11
2.4.2 Programming Languages & Markup	11
2.4.3 Backend & API Integration	12
2.4.4 Libraries & Dependencies	12
2.5 Software Configuration	13
2.5.1 Gradle Build System	13
2.5.2 Git and GitHub (Version Control System)	13
2.5.3 Postman (API Testing & Documentation)	14
2.6 Feasibility Analysis	14
2.7 Use Case Scenarios	14-15

CHAPTER 3 Software Design	16-27
3.1 System Architecture	16-17
3.1.1 Client Side	16
3.1.2 Server Side	16
3.2 Design Approach	17
3.3 User Interface Design	18-20
3.3.1 Design Principles	18
3.3.2 UI Components	19
3.3.3 User Navigation & Task Flow	19
3.3.4 Visual Style & Branding	20
3.4 Data Flow Diagram	21-23
3.4.1 Level 0 DFD	21
3.4.2 Level 1 DFD	21
3.4.3 Textual Representation of DFD	23
3.5 Use-Case Diagram	23-24
3.6 Database Design	25
3.6.1 Entity Diagram	25
3.6.2 Table	26
3.7 Design Considerations	26-27
 CHAPTER 4 Implementation and User Interface	 28-34
4.1 Implementation	28-31
4.1.1 Development Environment and Tools	28
4.1.2 Core Modules	28
4.1.3 Integration	30
4.2 User Interface	31-34
4.3 Output Screen	34
 CHAPTER 5 Software Testing	 35-40
5.1 Introduction	35
5.2 Testing Objectives	35
5.3 Testing Methodologies	35-36
5.3.1 White Box Testing	35
5.3.2 Black Box Testing	36
5.4 Levels of Testing	36
5.4.1 Unit Testing	36
5.4.1 Integration Testing	36
5.4.1 System Testing	36
5.4.1 User Acceptance Testing	37
5.5 Test Cases	37

5.6	Testing Tools	37
5.7	Errors and Debugging	39
	5.7.1 GPS Accuracy	39
	5.7.2 Voice Input Misinterpretation	39
5.8	Debugging Tools Utilized	40
5.9	Result Analysis	40
5.10	Conclusion	40
CHAPTER 6 Snapshots		41-49
CHAPTER 7 Conclusion		50
CHAPTER 8 Summary		51-52
APPENDICES		53-54
REFERENCES		55

List of Figures

Figure 3.4.1
Figure 3.4.2
Figure 3.5
Figure 3.6.1
Figure 6.1.1: Home

List of Tables

Table 1.3.4
Table 3.6.2
Table 4.2
Table 5.5

CHAPTER -1

INTRODUCTION

1.1 Overview and Motivation

The rapid horizontal expansion of modern educational institutions has transformed university campuses into "mini-cities." GLA University, Mathura, spanning over 110 acres with dozens of academic blocks, laboratories, hostels, and administrative wings, is a prime example of such growth. While this expansion is a testament to academic progress, it introduces a significant logistical challenge: Navigational Complexity.

For a first-year student, a visiting parent, or a guest lecturer, the task of finding a specific room in "Block 11" or locating the "Department of CS&IT" can lead to "transit anxiety" and loss of productive time. Conventional navigation tools like Google Maps often fail to provide the granular, "building-level" detail required for campus navigation. They typically show the university as a single point or a cluster of large shapes without clearly defined internal pathways or room-specific markers.

The motivation behind **GLA Routes** stems from the need for a localized, geospatial solution. The goal was to create a digital companion that does more than just show a path; it integrates the entire campus ecosystem—transport, academic data, and visual orientation—into a single, high-performance Android interface. By leveraging the power of Java and Cloud Integration (Firebase), this project aims to digitize the campus experience, supporting the broader vision of a "Smart Campus" where information and location are available at the tap of a screen.

1.2 Objective

The development of GLA Routes is guided by a multi-faceted approach aimed at solving the "Spatial Navigation Problem" inherent in large-scale academic environments. The primary objective is to engineer a seamless, real-time navigational ecosystem that bridges the gap between digital data and physical campus geography.

The detailed objectives are as follows:

1.2.1 Advanced Geospatial Navigation Development

- **Interactive Campus Mapping:** To design and architect a specialized Android-based mobile application that provides a high-fidelity, interactive overlay of the GLA University campus. This goes beyond generic maps by labeling specific departmental blocks (e.g., Block 9, Block 11) and internal landmarks that are traditionally ignored by commercial GPS systems.
- **Precision Pathfinding Algorithms:** To implement the most efficient routing logic that calculates the shortest distance from the user's real-time GPS coordinates to their selected destination. This involves utilizing the Google Maps Polyline API to draw clear, navigable paths that avoid confusion and minimize travel time.

1.2.2 Backend Architecture & Data Integrity

- **Robust Cloud Integration (Firebase):** To utilize Firebase as the primary backend-as-a-service (BaaS). This includes using the Firebase Realtime Database to store precise latitude and longitude coordinates for every significant campus node, ensuring that the navigation remains accurate even if building names or functions change over time.
- **Secure Authentication Framework:** To establish a secure user-authentication gateway. This ensures that features like "Ride Booking" are tied to verified student/faculty identities, maintaining the security and integrity of university-provided services.

1.2.3 Information Accessibility & Student Utility

- **Centralized Campus Search Engine:** To engineer a comprehensive search module capable of indexing diverse campus locations including:
 - **Academic Wings:** Department-specific blocks and laboratories.
 - **Utility Centres:** Canteens, libraries, and administrative offices (Registrar, Accounts, etc.).

- **Dynamic Information Portal:** To extend the app's functionality beyond navigation by integrating a "Student Information Hub." This provides users with instant access to academic fee structures, placement records, and campus event schedules, eliminating the need for multiple disparate web searches.

1.2.4 User-Centric Design and Orientation

- **Virtual Architectural Orientation:** To integrate a 360-degree Panoramic Viewer. This objective aims to reduce "arrival anxiety" by allowing users to visually inspect the building facade and entrance of their destination through high-resolution images and immersive views before they physically reach the location.
- **Heuristic UI/UX Optimization:** To design a user interface based on modern material design principles. The goal is to ensure that even a first-time visitor with minimal technical knowledge can navigate the app's features—ranging from "Ride Booking" to "Map Filtering"—with zero learning curve.

1.2.5 Institutional Impact and Efficiency

- **Smart Campus Evolution:** To contribute to GLA University's digital transformation. By automating the campus ride-booking process and digitizing navigation, the app reduces the workload on physical help-desks and improves the overall efficiency of campus transit systems.
- **Enhanced Inclusion:** To improve campus accessibility for differently-abled individuals or visitors who may find the vast layout of GLA University challenging to navigate without digital assistance.

1.3 Summary and Comparative Analysis of Similar Applications

In the current technological landscape, geolocation and navigation services have reached a high state of maturity. However, a significant "Information Gap" exists between macro-level city navigation and micro-level institutional wayfinding. This section analyses prominent existing solutions and identifies the specific deficiencies that GLA Routes addresses.

1.3.1 Google Maps

Google Maps is the industry standard for global positioning and vehicular navigation. While it offers unparalleled data for city streets and highways, its utility diminishes significantly within a private institutional boundary like GLA University.

- **Limitations:**
 - Inadequate Internal Routing: Google Maps often treats a university as a single polygonal entity. It rarely recognizes internal pedestrian-only pathways, leading users to "dead ends" or fenced boundaries.
 - Lack of Granularity: It does not identify specific "Blocks" (e.g., Block 1, Block 9) or internal departments by their local academic nomenclature.
 - Commercial Noise: The interface is often cluttered with external businesses, which distracts a student trying to find a specific lab or classroom.

1.3.2 Here WeGo and OpenStreetMap (OSM)

These platforms provide robust offline mapping capabilities and are favored for their community-driven data.

- **Limitations:**
 - Data Lag: Updates to university infrastructure (newly constructed wings or temporary roadblocks) take months or years to reflect on these global platforms.
 - Missing Utility Integration: These apps are purely navigational. They cannot provide campus-specific services like ride booking, fee details, or placement analytics, which are core to the GLA student experience.

1.3.3 Specialized Campus Navigation Systems

Several Tier-1 institutions (such as IITs or Stanford) have developed proprietary "Campus Map" applications.

- **Limitations:**

- **Non-Transferability:** These applications are hard-coded for specific geographic coordinates and server architectures, making them unusable for GLA University.
- **Static Nature: Most of these apps function as "Digital PDF Maps." They lack the real-time** interactivity provided by GLA Routes, such as live GPS tracking and dynamic ride-booking.

1.3.4 Comparative Analysis Table

To visualize the necessity of this project, the following table compares GLA Routes with general-purpose navigation tools:

Table 1.3.4 Comparison Table

Feature	Google Maps	Here WeGo	GLA Routes
Global Coverage	Excellent	Good	Campus Specific
Internal Block Labelling	Minimal	None	High Precision
Pedestrian Pathways	Partial	Limited	Fully Mapped
Ride Booking Integration	External (Uber/Ola)	None	Internal (GLA Ride)
360° Architectural View	Street View (Limited)	None	Custom Integrated
Academic Info Hub	No	No	Yes (Fees/Placement)

1.3.5 The "GLA Routes" USP (Unique Selling Proposition)

Unlike the aforementioned applications, GLA Routes is architected as a Socio-Technical System. It recognizes that a student's journey doesn't end at the gate; it begins there. By

focusing on "Block-wise" navigation and integrating the Firebase Realtime Database, the app ensures that the data is:

1. **Hyper-Local:** Every marker is placed based on physical verification of GLA campus coordinates.
2. **Service-Oriented:** It bridges the gap between "Where I am" and "What I need" (e.g., booking a ride or checking fees).
3. **Visual-First:** Using the 360-degree view helps newcomers overcome the "recognition gap" when facing similar-looking academic buildings.

1.4 Organization of the Project

This project report is organized into chapters, each describing a major stage of system development:

1.Introduction

Provides an overview of the project, including motivation and objectives

2. Software Requirement Analysis

Describes functional and non-functional requirements, along with feasibility analysis

3.Software Design

Explains system architecture, data flow diagrams, module design, and user interface structure

4.Implementation and User Interface

Covers the development of the application using Android Studio, Java, XML, Google Maps API, and Firebase, along with UI screens

5.Software Testing

Describes testing methodologies, test cases, results, and validation techniques

6.Conclusion

Summarizes the project outcomes and overall achievements

7.Summary

Highlights key learnings and provides insights into future enhancements

Additional sections such as appendices and documentation provide supporting details related to the project.

CHAPTER -2

Software Requirement Analysis

Software Requirement Analysis

The Software Requirement Analysis (SRA) phase is the most critical step in the Software Development Life Cycle (SDLC). For GLA Routes, this process involved a systematic evaluation of the needs of students, faculty, and visitors at GLA University. This chapter delineates the functional boundaries, performance constraints, and resource prerequisites necessary to ensure the application's success as a reliable campus utility.

2.1 Functional Requirements

Functional requirements define the specific behaviours and services the system must provide. For GLA Routes, these are focused on navigation accuracy and data retrieval:

1. **Campus Map Visualization:** The system must render a high-definition, interactive map of the GLA University campus using the Google Maps SDK, ensuring all internal roads and landmarks are visible.
2. **Location Search Engine:** A robust search functionality must be provided, allowing users to input strings (e.g., "Block 9", "Canteen", "CSED") and receive matching results indexed from the backend.
3. **Dynamic Route Calculation:** The application shall interface with the Google Directions API to calculate and draw a polyline representing the shortest navigable path between two points.
4. **GPS Synchronization:** The system must request and maintain real-time access to the device's GPS hardware to update the "User Marker" as they move through the campus.
5. **Cloud Data Persistence:** Firebase must act as the "Source of Truth," storing the specific User's details that is name, email and phone.

6. **Interactive UI Controls:** The system must support standard touch gestures, including pinch-to-zoom and panning, to allow users to inspect different zones of the campus with ease.

2.2 Non-Functional Requirements

Non-functional requirements describe "how" the system should perform. While functional requirements define the tasks, non-functional requirements define the experience, ensuring the application is professional, reliable, and production-ready.

- **Performance:** * The application must ensure a **"First Meaningful Paint"** (fully interactive map) within 3 seconds of launch on 4G/LTE networks.
 - API response times for fetching "Ride Booking" status or "Placement Info" must not exceed 500ms to prevent UI stutter.
 - Memory consumption should be optimized to avoid background termination by the Android OS on devices with 4GB RAM or less.
- **Usability:** * Following the principle of **"Minimalism"** and **"Fitts's Law,"** primary action buttons (like "Get Map") must be large enough for easy thumb interaction.
 - The navigation flow must be optimized so that a user can start route guidance in **no more than three taps** from the home screen.
 - Visual feedback, such as loading shimmers or progress bars, must be present during all data-fetching operations.
- **Reliability & Availability:** * The system aims for **99.9% availability** by utilizing Firebase's globally distributed infrastructure.

- **Fault Tolerance:** In the event of a GPS signal loss (Indoor/Tunnels), the app must gracefully notify the user via a "Toast" message and switch to the last known coordinate rather than crashing.
- **Offline Persistence:** Basic block information and previously searched routes should remain accessible even if the user temporarily loses internet connectivity.
- **Security & Privacy:** All sensitive user data (Name, Email, Roll Number) must be encrypted in transit using **SSL/TLS 1.3** protocols.
 - **Authentication:** Unauthorized access to the "Admin" or "Driver" portals must be prevented through Firebase Security Rules, ensuring users can only read/write their own data.
 - **Least Privilege:** The application should only request "Fine Location" permissions when the map module is active to protect user privacy.
- **Scalability:** The Firebase Realtime Database must use a "Flat JSON" structure to accommodate the addition of new blocks, hostels, or canteens without requiring a code overhaul.
 - **Concurrent Users:** The backend must support at least 500 concurrent users (simultaneous register) during peak hours.
- **Portability:**
 - The application must be compatible with **Android 7.0** and above, ensuring coverage for over 90% of currently active Android devices in the student population.
 - The layout must be adaptive (using dp and sp units) to ensure consistent rendering across different screen densities (HDPI, XHDPI, XXHDPI).

2.3 Hardware Requirements

The hardware environment is a critical factor in both the development lifecycle and the end-user experience. The following requirements ensure that the development environment remains stable and that the mobile client can handle intensive tasks like real-time GPS tracking and 360° image rendering.

1. Development Environment (Workstation)

To handle the heavy resource demands of **Android Studio** and the **Gradle** build system, the following workstation specifications are required:

- **Processor:** Intel Core i5 (8th Gen or higher) or AMD Ryzen 5. A multi-core processor is essential for parallelizing code compilation and reducing build times.
- **Memory (RAM): Minimum 8 GB (16 GB Recommended).** Higher RAM allows the developer to run the Android Emulator, a web browser for Firebase Console, and the IDE simultaneously without system throttling.
- **Storage: Minimum 256 GB SSD.** A Solid State Drive (SSD) is mandatory for rapid indexing of SDK files and faster deployment of APKs to testing devices.
- **Display:** Minimum 1366x768 resolution. High-resolution displays are preferred for detailed UI/UX design work in the Android XML Layout Editor.

2. Test Environment (Mobile Device)

Testing on physical hardware is necessary to validate sensor performance that cannot be accurately simulated.

- **Operating System:** Android 8.0 (Oreo) or higher. This ensures compatibility with modern Google Play Services and Firebase SDK libraries.
- **Sensors: GPS/GNSS:** Required for real-time location accuracy.
- **Network:** 4G/LTE or high-speed Wi-Fi capability. Stable connectivity is required for real-time data syncing with the Firebase database and fetching map tiles from the Google Cloud.

3. Infrastructure & Connectivity

- **Stable Broadband Connection:** A minimum speed of 2 Mbps is required for initial SDK downloads and consistent API handshaking with the Google Maps Directions API.
- **USB Data Cable:** A high-quality USB 3.0/Type-C cable for ADB (Android Debug Bridge) to allow fast code deployment and real-time log monitoring during physical device testing.

2.4 Software Requirements

The software requirements define the integrated development environment (IDE), programming languages, and third-party services necessary to compile and run the application. A precise software stack is crucial for ensuring version compatibility and API stability.

2.4.1 Development Software (IDE & Environment)

- **Operating System:** Windows 10/11 (64-bit), macOS (11.0 Big Sur or higher), or Linux (Ubuntu 20.04+).
- **Integrated Development Environment (IDE):** Android Studio (Electric Eel | 2022.1.1 or later). This provides the essential tools for code editing, UI layout design, and real-time debugging.
- **Java Development Kit (JDK):** JDK 11 or 17. Modern Android development requires these versions to support current Gradle build tools and language features.
- **Android SDK:** Target API Level 33 (Android 13) or higher, with Support Libraries back-dated to API Level 26 (Android 8.0) to ensure a wide installation base.

2.4.2 Programming Languages & Markup

- **Java/Kotlin:** The primary logic of the application is written in **Java** (or Kotlin), handling the heavy lifting of GPS data processing and Firebase communication.

- **XML (Extensible Markup Language):** Used for designing the user interface. This includes the layout files for the 80% Map view, custom dashboard cards, and the registration forms.
- **JSON (JavaScript Object Notation):** The data format used for communicating with the Firebase Realtime Database and the Google Maps API.

2.4.3 Backend & API Integration

- **Firebase Realtime Database:** A NoSQL cloud-hosted database used to store and sync student names, roll numbers, and ride booking locations in real-time.
- **Firebase Authentication:** Handles secure user sign-in and account creation using email/password verification.
- **Google Maps SDK for Android:** The core engine used to render the university map, place custom markers, and handle user interaction (zoom/tilt).
- **Google Directions API:** Utilized to calculate and draw the **Polyline** routes between various campus blocks.

2.4.4 Libraries & Dependencies

- **Glide :** Image loading libraries used to render building photos and 360° gallery thumbnails efficiently without causing memory leaks.
- **Google Play Services:** Required on the mobile device to enable Location Services and Map rendering.
- **Material Design Components:** Used to implement modern UI elements like "Category Chips," "Floating Action Buttons," and "CardViews."

2.5 Software Configuration Management

Software Configuration Management is a critical discipline used to establish and maintain the consistency of the GLARoutes application's performance and functional attributes. By utilizing a standardized SCM stack, the project ensures that code remains modular, reproducible, and protected against data loss.

2.5.1 Gradle Build System

Gradle serves as the advanced build toolkit for the application, automating the transformation of source code and resources into a deployable Android Package (APK).

- **Dependency Management:** Automates the inclusion of external libraries such as `com.google.android.gms:play-services-maps` and `com.google.firebase:firebase-database`. This ensures that all developers are using synchronized versions of third-party APIs.
- **Build Variants:** Configured to manage different versions of the app (e.g., a "Debug" version with logging enabled and a "Release" version optimized for the Play Store).
- **Automated Signing:** Handles the secure signing of the APK with a keystore file, which is a prerequisite for distribution.

2.5.2 Git and GitHub (Version Control System)

Version control is the backbone of the project's development, providing a chronological record of all changes made to the codebase.

- **Granular Commits:** Development was broken down into logical "commits." For instance, adding the **360° View** was committed separately from the **Ride Booking** logic, allowing the team to revert to a previous stable state if a new feature introduced a regression.
- **Branching Strategy:** Utilized a branching model where the "Master" branch remained production-ready, while "Feature" branches were used for experimental UI layouts (like the 80% map-to-screen ratio).
- **Remote Repository:** GitHub acted as a secure cloud backup, ensuring that the source code remains accessible even in the event of hardware failure on the development workstation.

2.5.3 Postman (API Testing & Documentation)

Before implementing complex networking logic in Java/Kotlin, Postman was used as an intermediary testing suite.

- **Endpoint Validation:** Used to send GET requests to the **Google Directions API** to verify that the API Key was active and that the university coordinates (Latitude/Longitude) returned valid route data.
- **JSON Schema Verification:** Verified the structure of the JSON response to map the correct keys (such as `overview_polyline`) for parsing within the mobile application.

2.6 Feasibility Analysis

Before development began, a four-pillar feasibility study was conducted:

1. **Technical Feasibility:** The team verified that the Google Maps API could handle custom markers for campus blocks. Since Android Studio and Firebase are natively integrated, the technical risk was deemed low.
2. **Operational Feasibility:** Observations of students at GLA University confirmed a high demand for navigation. The app requires no training, making it operationally sound for the target demographic.
3. **Economic Feasibility:** This is an open-source/student project. Since the Google Maps API and Firebase offer a generous "Free Tier" for development, the project involves zero initial capital expenditure.
4. **Legal/Time Feasibility:** The project respects privacy laws by not storing sensitive GPS history. The timeline was divided into sprints (Design, Dev, Test), ensuring completion within the academic semester.

2.7 Use Case Scenarios

Use Case 1: Search for a Campus Location

Actor: User (Student/Faculty/Visitor)

Scenario: 1. The user opens the GLARoutes App.

2. The user enters the name of a campus location (e.g., "Block 1").

3. The system fetches the location from Firebase.
4. The app highlights the selected location on the campus map.

Use Case 2: View Details of a Campus Building

Actor: User

- Scenario:**
1. The user taps on a block button on the map.
 2. The system retrieves stored data.
 3. Details such as building name, department, or services are displayed.

Use Case 3: Finding the Shortest Route (Detailed Flow)

- **Actor:** A visitor arriving at the Main Gate.
- **Pre-condition:** App is installed and GPS is enabled.
- **Main Flow:**
 1. The user searches for "Block 1."
 2. The app calculates the distance from "Main Gate" (current location) to "Block 1."
 3. A blue polyline appears on the map.
 4. The user begins walking; the app updates the "User Dot" in real-time.
- **Post-condition:** User reaches the destination within the estimated time.

Use Case 4: Real-Time Location Tracking

- **Actor:** User
- **Scenario:**
 1. The user enables GPS and opens the navigation screen.
 2. The app continuously fetches updated location coordinates.
 3. The Google Maps API refreshes movement on the map.
 4. The user sees their real-time movement on the campus map.

CHAPTER -3

Software Design

The **software design** of *GLARoutes* focuses on creating a scalable, user-friendly, and efficient Android-based navigation system tailored for GLA University. The design follows a modular and layered architecture to ensure maintainability, flexibility, and ease of future enhancements.

3.1 System Architecture

GLA Routes utilizes a **Client-Server Architecture** optimized for mobile environments. This separation ensures that the heavy lifting of data storage and route indexing is handled by the server (Firebase), while the mobile device focuses on rendering the user interface and processing GPS signals.

3.1.1 Client Side (Android Application)

The front-end is a native Android application developed using **Java and XML**.

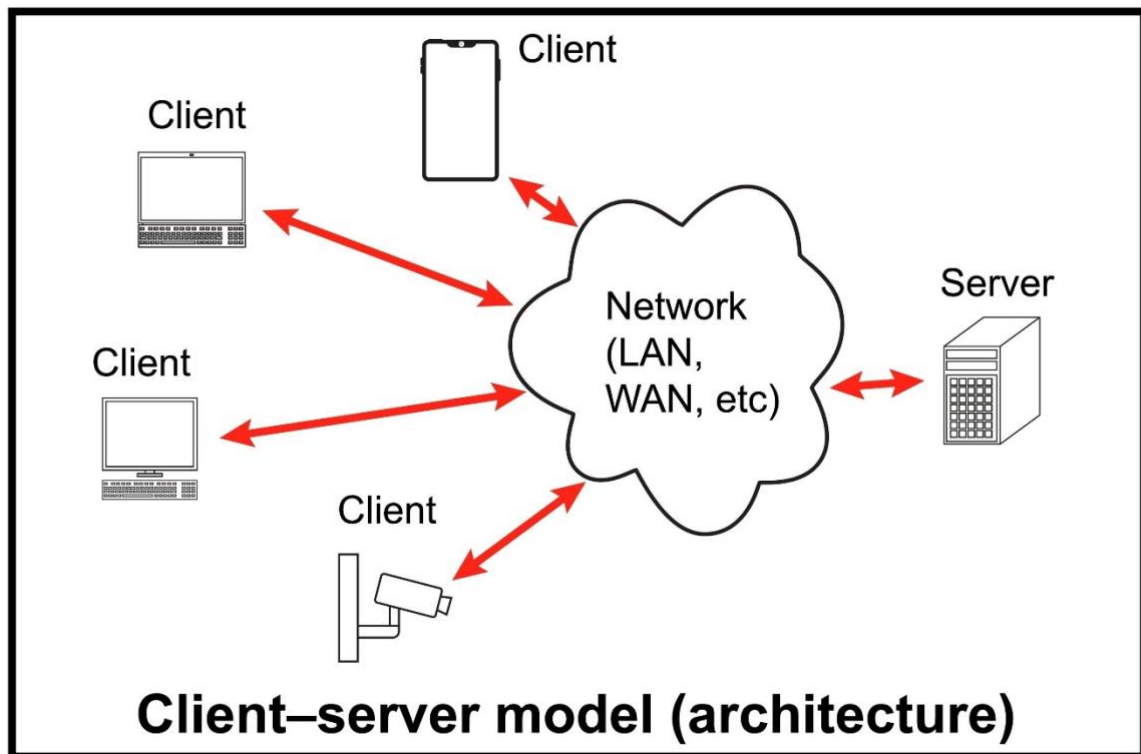
- **UI/UX Layer:** Defined in XML, this layer provides the visual components like the Google Map fragment, search bars, and floating action buttons.
- **Logic Layer:** Handled in Java, this manages the Activity Lifecycle, permission handling (GPS), and API calls.
- **Input Handlers:** The app supports multimodal input, including standard text queries and **Google Voice Search integration** for hands-free navigation.

3.1.2 Server Side (Firebase Database)

The backend leverages the **Firebase Realtime Database**, a NoSQL cloud-hosted database.

- **Data Persistence:** It stores details of every user that registered on the app.
- **Real-time Sync:** When a user books a ride or a location is updated, the changes are pushed to all clients instantly without requiring a page refresh.

Figure 3.1.2 Client-Server Architecture



3.2 Design Approach (MVC Pattern)

The application follows the **Model-View-Controller (MVC)** design pattern to organize code efficiently:

- **Model:**

This represents the "Knowledge" of the system. It consists of Java classes that define "Blocks," "Users," and "Ride Requests." It communicates directly with Firebase to fetch raw data.

- **View:**

This is the visual representation of the Model. In GLA Routes, the View includes the Activity_main.xml, the Google Map markers, and the custom Info Windows that appear when a block is tapped.

- **Controller:**

The Controller acts as the brain. It listens for user actions (like clicking "Find Route"), requests the coordinates from the Model, and instructs the View to draw the Polyline on the map.

This approach improves code readability, maintainability, and reusability.

3.3 User Interface Design

3.3.1 Design Principles

The architecture of the GLARoutes interface is governed by six core pillars intended to optimize student productivity and campus accessibility:

- **Simplicity:** By adopting a "Content-First" approach, the interface removes cluttered sidebars and intrusive pop-ups, ensuring that even a first-time visitor can find their destination within two taps.
- **Consistency:** The application uses a unified Design Language System (DLS). Whether a user is checking "Placement Info" or "Course Fees," the typography, padding, and icon sets remain identical, reducing the cognitive load required to learn the app.
- **Responsiveness:** The UI is built using Constraint Layout, allowing it to scale seamlessly across various screen sizes and orientations (Portrait for navigation, Landscape for 360° views) without losing element alignment.
- **Accessibility:** Beyond standard touch input, the app supports Voice-to-Text Search and high-contrast colour modes, ensuring that users with visual impairments or those carrying heavy bags can navigate the campus safely.
- **Visual Hierarchy:** Elements are weighted by importance. The Map Fragment occupies 80% of the screen real estate, while secondary information like "Block Descriptions" is housed in expandable bottom sheets that only appear when requested.
- **Feedback Loops:** Every user interaction is acknowledged. Visual states like *Ripple Effects* on buttons, *Progress Bars* during data fetching, and *Toast Notifications* for successful Firebase operations ensure the user is never left wondering if an action worked.

3.3.2 UI Components

The application's toolkit consists of specialized Android components designed for a campus ecosystem:

- **Persistent Search Bar:** A top-anchored Search View that remains visible across the Map module, allowing for "instant-jump" navigation to any building or Academic blocks.
- **Interactive Map View:** A Google Maps implementation featuring custom markers, dynamic zoom levels, and the "Get Map" tracking button.
- **Category Chips:** Material Design **Filter Chips** located below the search bar. These allow for one-tap filtering of the map to show only specific block.
- **Minimalist Dashboard:** A grid-based hub that utilizes Card View components with subtle shadows to distinguish between different services like Placements, 360 degree View, and Ride Booking.
- **Firebase Integration Layer:** A background logic component that ensures the UI is always "Fresh".

3.3.3 User Navigation & Task Flow

To ensure the app serves its purpose efficiently, the UI follows a logical task-based flow:

- **Primary Path (Navigation):** *Login* → *Home* → *Dashboard* → *Block List* → *Map Interface*. This path is optimized for speed to help students reach classes on time.
- **Secondary Path (Utility):** *Menu* → *Clubs/Fees/Events/Library*. This path uses list-based views for high-density information reading.
- **The "Emergency" Path:** *Dashboard* → *Ride Booking*. A simplified form-based interface designed for quick data entry and immediate feedback.

3.3.4 Visual Style & Branding

- **Color Palette:** The app utilizes the official University colours (Blue and White) to foster a sense of belonging and officiality.
- **Typography:** A sans-serif font family (like **Roboto**) is used for its high legibility on mobile screens, especially when reading small text like "Room Numbers" or "Course Codes."
- **Iconography:** Linear, minimalist icons are used for the Menu (Library, Club, Events) to keep the interface modern and professional.

3.4 Data Flow Diagram

The Data Flow Diagram (DFD) of the application illustrates how user data moves through the system during the registration process.

When a new user registers in the application, they are required to enter their personal details such as Full Name, Email, and Phone Number. This information is captured through the user interface (mobile app).

Once the user submits the registration form:

1. The data is sent to the application backend.
2. The backend processes and validates the entered details.
3. After successful validation, the data is securely stored in the Firebase Realtime Database.
4. Each user is stored with a unique ID, and their details are organized under the Users node in the database.

The system allows:

- Storage of user information for future access
- Retrieval of user data when required (e.g., login or profile access)

Main Components in DFD:

- External Entity: User
- Process: Registration & Data Validation
- Data Store: Firebase Realtime Database
- Data Flow: User Details (Name, Email, Phone)

Flow Summary:

User → Registration Form → Data Validation → Firebase Database → Stored User Record

3.4.1 Level 0 DFD

The Level 0 Data Flow Diagram represents the overall working of the system in a simplified manner. It shows how the **User** interacts with the **User Registration System**.

- The user enters registration details (Full Name, Email, Phone).
- The system processes this information.
- The data is stored in the **Firestore Realtime Database**.
- The system returns a response (Success/Failure) to the user.

Components:

- **External Entity:** User
- **Process:** User Registration System
- **Data Store:** Firestore Realtime Database
- **Data Flow:** User Details & Registration Status

Level 0 Data Flow Diagram



Figure 3.4.1 DFD 0

3.4.2 Level 1 DFD (Process Decomposition)

The Level 1 Data Flow Diagram provides a detailed breakdown of the registration process.

Step-by-step flow:

1. **Capture User Details (1.1):**
The system collects user input (Name, Email, Phone).

2. **Validate User Details (1.2):**
 - Checks if data is correct and complete.
 - If invalid → goes to error process.
3. **Show Error Message (1.3):**
 - Displays error and asks user to re-enter details.
4. **Store User Details in Firebase (1.4):**
 - Valid data is stored in Firebase Realtime Database.
5. **Registration Success:**
 - Confirmation message is shown to the user.

Components:

- **Processes:** Capture, Validate, Store, Error Handling
- **Data Store:** Firebase Realtime Database
- **External Entity:** User
- **Data Flow:** Input Data, Validated Data, Error Messages, Success Response

Level 1 DFD

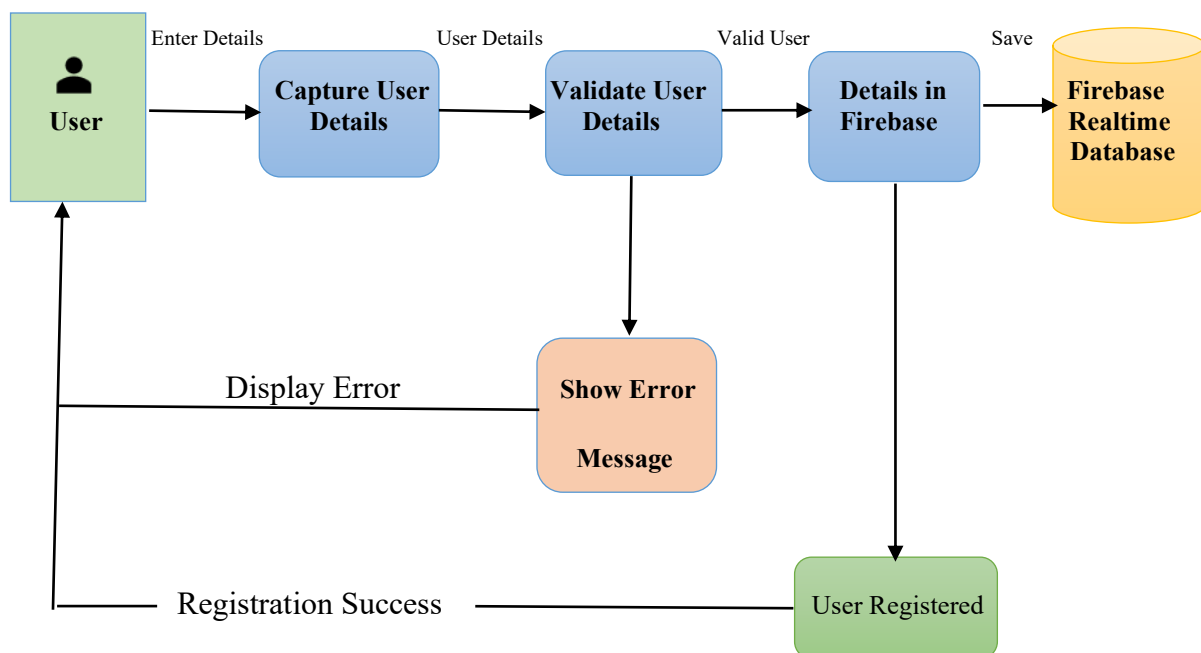


Figure 3.4.2 DFD 1

3.4.3 Textual Representation of DFD

User → Input(details) → Controller → Firebase Database → Route Calculation
→ Acknowledgement → User

3.5 Use Case Diagram

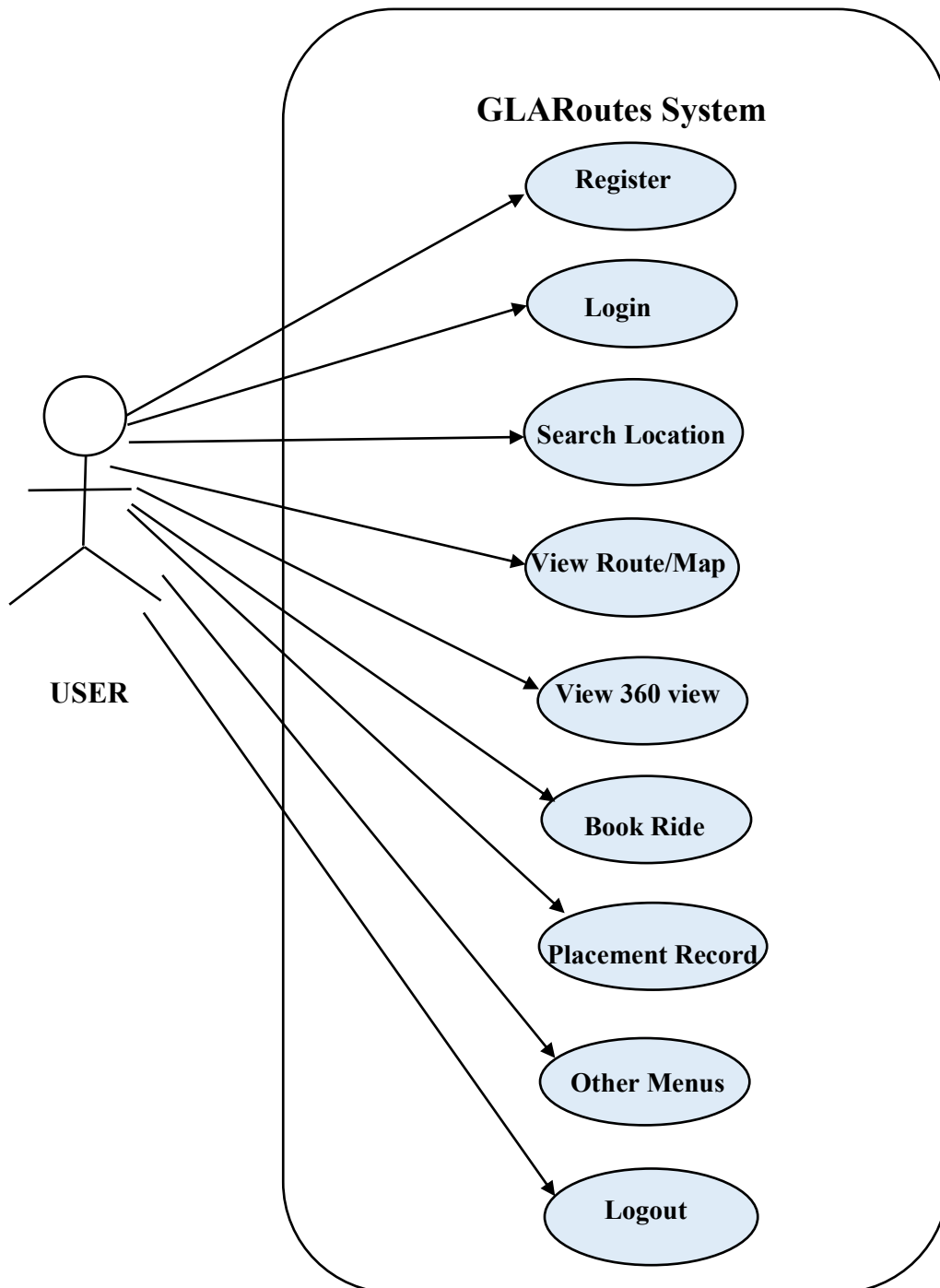


Figure 3.5 Use Case

Use Case Actor: User

The *User* is the primary actor who interacts with the GLARoutes system to perform various operations such as registration, login, navigation, and viewing location details.

User Use Cases:

1. Register

- User enters Full Name, Email, Phone
- System validates and stores data in Firebase

2. Login

- User enters credentials
- System verifies and grants access

3. Search Location / Get Directions

- User searches for a destination inside campus
- System processes and finds the best route

4. View Route on Map

- User views navigation path
- System displays route using map

5. View 360° Location

- User clicks 360° view button
- System shows panoramic view of selected place

6. View Placement Records

- User checks placement details
- Course wise data available

7. View Other Menus

- User can view other menus
- Help, Library ,Course , Event details.

3.6 DATABASE DESIGN

3.6.1 Entity Diagram

- The database design for the system is minimal and consists of a single entity, **Users**. This entity stores essential information about all types of users, including students, faculty, and visitors.
- Each user is uniquely identified by a **user_id**, which serves as the primary key. The attributes such as **name**, **email**, and **phone number** are used to store basic user details required for authentication and communication.
- User Entity: Represents the students and faculty registered on the app.
 - Attributes: Name, Email, Phone, id.

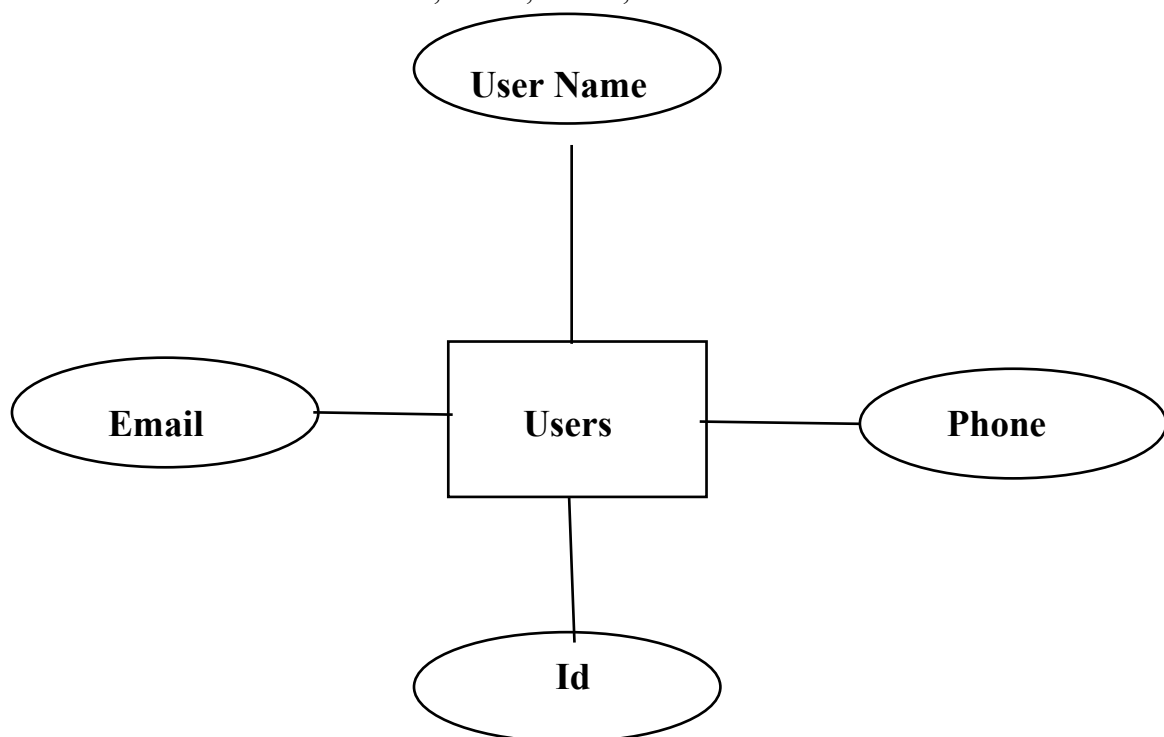


Figure 3.6.1 Entity Diagram

3.6.2 Table

Table 3.6.2 User Registration Table

Field Name	Data Type	Description
name	String	User
phone	Number	Phone number
email	String	User Email
id	String	Unique Key

3.7 Design Considerations

1. Accuracy: Precision-First Mapping

Accuracy in a campus environment is measured in meters. To ensure students don't end up on the wrong side of a wall:

- **Coordinate Precision:** Using high-resolution Latitude and Longitude coordinates (up to 6 decimal places) for every entry point of buildings like CSED or the Engineering Wing.
- **Map Type Optimization:** Utilizing View overlays during the development phase to verify that digital markers align perfectly with physical rooftops and footpaths.

2. Performance: Low Latency, High Speed

Performance is critical for users on the move who may have varying network strengths.

- **Realtime Synchronization:** By leveraging Firebase Realtime Database, data changes (like a shuttle's location) are pushed to the client in milliseconds.

3. Scalability: Future-Proof Architecture

The university is constantly evolving; the app must do the same without a full code rewrite.

- **Modular Database Schema:** The NoSQL structure in Firebase allows for User objects to be access dynamically.
- **Component-Based UI:** Using Fragments and RecyclerViews in Android Studio so that new utility modules (e.g., "Hostel Management") can be plugged into the main navigation drawer with minimal friction.
- **API Agnostic Design:** The architecture is built to support future integrations, such as adding AR Core for augmented reality navigation or third-party payment gateways for fee management.

4. Usability: User-Centric Design (UCD)

A map is useless if it's too cluttered to read.

- **Accessibility Features:** Voice-to-Text: Integration of Google's Speech Recognition for hands-free navigation.
- **Color Contrast:** High-contrast markers and clear typography to assist users with visual impairments or those viewing the screen in direct sunlight.
- **Intuitive Feedback:** Using Toasts to provide immediate confirmation when a user interacts with the "Free Ride" or "Placement" modules.

5. Reliability: Fault-Tolerant Operations

Reliability ensures the app works even when the environment doesn't.

- **Offline Persistence:** Enabling Firebase's `setPersistenceEnabled(true)` feature. This allows students to view previously loaded maps and building info even if they lose connectivity in basement labs or high-interference areas.

- **Graceful Degradation:** If the Google Maps API fails to load, the app is programmed to switch to a simplified "List View" of campus directories, ensuring the user is never left without information.

CHAPTER - 4

Implementation & User interface

4.1 Implementation

The implementation of **GLARoutes** involves a robust integration of front-end components, back-end services, and Google Cloud APIs to provide a seamless navigation experience.

4.1.1 Development Environment and Tools

The development environment included the following tools and platforms:

Software Tools

Android Studio (latest version) – Primary IDE for development and debugging.

Java Development Kit (JDK 8+) – Core programming language support.

Google Maps API Platform – Handles map rendering, geocoding, and route calculation.

Firebase Console – Manages Authentication, Realtime Database, and Cloud Storage.

Android Emulator – Used for real-time debugging and UI responsiveness testing.

XML Editor– Used for designing layouts and user interface components.

4.1.2 Core Modules

1. User Authentication Module

- Login and secure access using Firebase Authentication
- Handles new users and returning users

2. Location Search Module

- Fetches building names and coordinates
- Displays matching results dynamically

3. Navigation & Route Module

- Generates shortest path from user's current location
- Displays turn-by-turn navigation

4. Real-Time Tracking Module

- Updates user's movement on the map
- Uses GPS and Google Maps API

5. 360° View Module

- Provides immersive campus visualization
- Implemented using WebView
- Supports device motion (gyroscope-based view)

6. Building Information Module

- Shows detailed info for each campus building

7. Course Module

- Displays course-related information
- Shows course name, code, and description

8. Help Module

- Provides user assistance
- Includes FAQs and basic support

9. Ride Booking: An internal transport request system for campus commuters.

10. Placement & Career Module: A dedicated section showing upcoming company visits, eligibility, and recruitment news.

11. Campus Utility Menu: A centralized hub providing access to:

- **Course & Fees:** Detailed breakdown of academic finances.
- **Library:** Status of books and digital resources.
- **Clubs & Events:** Updates on technical fests, cultural programs, and workshops.

4.1.3 Integration

The integration phase is the "glue" that binds the front-end user interface with the cloud back-end and hardware sensors. The integration of **GLARoutes** is divided into three primary layers:

A. Cloud-to-App Integration (Firebase & APIs)

The application acts as a real-time client to Google's cloud infrastructure:

- **Authentication Flow:** The Registration Form is integrated with Firebase Auth. Upon successful registration, a unique UID is generated, which links the user's profile to their specific ride bookings and fee records.
- **API Orchestration:** The Google Maps SDK is integrated with the Directions API. When a user clicks "Get Map" on a Block Information card, the app passes the destination coordinates to the API, which returns a JSON response containing the polyline path and estimated travel time.

B. Hardware & Sensor Integration

GLARoutes leverages the physical hardware of the Android device to enhance navigation:

- **GPS & Fused Location Provider:** The Navigation module integrates with the device's GPS sensor. We use the FusedLocationProviderClient to balance accuracy and battery consumption, ensuring the user's "blue dot" moves smoothly across the campus map.
- **Gyroscope & Accelerometer:** The **360° View Module** integrates the device's motion sensors. By capturing the pitch, roll, and yaw of the phone, the app translates physical movement into virtual camera rotation within the WebView panorama.

C. Internal Module Communication (Java & Intents)

The various screens (Placement, Library, Fees, Map) communicate using structured Java logic:

- **Intent Passing:** When moving from the **Dashboard** to the **Placement Screen**, "Intents" are used to carry user session data so the student doesn't have to log in repeatedly.
- **Shared Preferences:** Essential user data (like Name and email) is stored locally using SharedPreferences. This allows the **Menu** (Library, Fees, Clubs) to display personalized information quickly without querying the database every time.
- **Adapter Patterns:** To integrate the **Clubs & Events** and **Course Fee** views,

4.2 User Interface

Table 4.2 User Interface

Component	Integration Point	Resulting Functionality
Registration Form	Firebase Auth + Database	Secure user profile creation and data persistence.
Block List	Google Maps API + GPS	Real-time pathfinding from current location to block.
Menu Button	Navigation Drawer	One-touch access to Library, Fees, and Club events.
360° Panorama	WebView + Gyroscope Sensor	Motion-controlled virtual campus tour.
Placement View	Placement Information Activity	Dynamic updates on company visits and job alerts.

The user interface is structured to minimize the "tap-distance" to essential campus information. It employs a consistent color palette and typography to maintain a unified brand identity for the university.

1.Home (Start) Screen

- The first screen displayed to the user
- Contains a “Explore Now” button
- Provides simple and clean design
- Navigates to Dashboard

2.Dashboard Screen

- Central hub of the application
- Provides access to all features
- Includes buttons/cards for:
 - Block Information
 - Ride Booking
 - Help
 - Courses
 - 360° View
 - Placement Record

3.Block Information Screen

- Displays list of academic blocks
- Each block contains:
 - Block name
 - Description
 - “Get Map” button

4. Map Screen

- Displays Google Map interface
- Shows:
 - Current location
 - Destination marker
 - Navigation route

5. Ride Booking Screen

- Form-based interface
- Allows user to:
 - Enter pickup location
 - Enter drop location
 - Submit ride request

6. Help Screen

- Provides assistance to users
- Includes FAQs and support information

7. Course Screen

- Displays course details
- Includes:
 - Course name
 - Course Fee
 - Description

8. Global Search Functionality

- **Instant Access System:** A persistent search bar or icon integrated into the Dashboard and list-heavy screens.
- **Search Capabilities:**
 - Predictive Text: Suggests blocks or courses the user types.

9. Placement Screen

- **Central Career Portal:** Provides a comprehensive overview of campus recruitment statistics and active opportunities.
- **Placement Record (Course-Wise):** A specialized data view that allows students to see historical performance based on their specific department.
- **Each Record Contains:**
 - **Course Selection Toggle:** Dropdown or tabs to switch between courses (e.g., B. Tech, MBA, MCA).
 - **Placement Percentage:** A visual progress bar showing the percentage of students placed in that course.

10. 360° View Screen

- Provides immersive campus experience
- Displays panoramic view
- Supports interaction through device movement

4.3 Output Screens

The following output screens are included in the application:

- Home Screen
- Dashboard Screen
- Block Information Screen
- Map Screen
- Ride Booking Screen
- 360° View Screen
- Search Results Overlay
- Placement Record Screen

Each screen is designed to provide a simple and user-friendly experience.

CHAPTER – 5

Software Testing

5.1 Introduction

Software testing is an essential phase in the development of the **GLARoutes** application. It ensures that the system functions correctly, meets user requirements, and is free from major errors. Testing helps in identifying bugs, improving performance, and ensuring reliability before deployment. For an application integrating real-time GPS and cloud-based authentication, testing verifies that the "Blue Dot" precision matches the physical campus layout and that user data is handled securely.

5.2 Testing Objectives

The quality assurance process for GLARoutes was driven by the following objectives:

- **Validation of Functional Flow:** To confirm that the transition from Splash Screen to Dashboard to Navigation is logical and error-free.
- **Database Accuracy:** To ensure that only specific data (User Name, Email and Phone) is recorded in Firebase during registration.
- **Latency Monitoring:** To measure the time taken to fetch 360° panoramic images.
- **Crash Resilience:** To identify edge cases where the app might fail, such as losing internet connectivity while on a live route.

5.3 Testing Methodologies

5.3.1 White-Box Testing

This involved testing the internal structures and workings of the application.

- **Path Testing:** Ensuring every conditional branch in the Java/Kotlin code (e.g., if (userLoggedIn)) was executed at least once.
- **API Handshaking:** Verifying the internal logic used to parse JSON responses from the Google Directions API.

5.3.2 Black-Box Testing

This focused on the functional requirements without looking at the internal code.

- **Boundary Value Analysis:** Testing the "Free Ride" limit. We tested if the app correctly stops allowing bookings after the 10th ride of the month.
- **Equivalence Partitioning:** Testing the search bar with various inputs (Building Names, Department Names, and Room Numbers) to ensure consistent results.

5.4 Levels of Testing

5.4.1 Unit Testing

Individual modules were validated to ensure they stand alone.

- **Registration Module:** Tested with invalid email strings (missing "@") to ensure the validator catches errors.
- **Distance Module:** Unit tested the formula used to calculate the "16.0 km" display to ensure it rounds decimals correctly.

5.4.2 Integration Testing

Since the app relies on third-party services, integration testing was paramount:

- **Firebase & Auth Integration:** Verified that a "Success" message only appears after the Firebase onCompleteListener returns a positive result.
- **Map & UI Integration:** Verified that clicking the "MAP" button on the Block 11 (CSED) card successfully passes the correct latitude and longitude to the Google Map intent.

5.4.3 System Testing (End-to-End)

The entire application was tested as a single entity on multiple devices:

- **Usability Testing:** Evaluated if the 80% Map view provided enough room for the "Explore GLA" popup without overlapping critical navigation controls.

- **Real-world GPS Testing:** Tested the app physically at the GLA University Main Gate to verify if the "Start Location" synced with the actual GPS coordinates.

5.3.4 User Acceptance Testing (UAT)

This testing is conducted with real users such as students and faculty members.

- Verifying ease of use
- Checking navigation accuracy
- Collecting user feedback

5.5 Test Cases

Table 5.5 Test Cases

<i>Test Case ID</i>	<i>Test Description</i>	<i>Expected Result</i>	<i>Status</i>
<i>TC1</i>	Search location using text	Correct location displayed	Passed
<i>TC2</i>	Search location using voice	Accurate conversion and result	Passed
<i>TC3</i>	Display current location	Correct real-time location shown	Passed
<i>TC4</i>	Route generation	Shortest path displayed	Passed
<i>TC5</i>	Map loading	Map loads without delay	Passed

5.6 Testing Tools

1. Device and Environment Simulation

- **Android Emulator & iOS Simulator:** Used for rapid iterative testing during development to simulate different screen densities, API levels (Android 10 to Android 16), and hardware configurations (e.g., notch displays, foldable screens).

- **Postman / Insomnia:** Utilized for **API Testing** to verify that backend endpoints return the correct JSON structures before they are integrated into the mobile UI.

2. Firebase Quality Suite

- **Firebase Test Lab:** A cloud-based infrastructure that runs the app on a wide range of real physical devices hosted in Google's data centers.
 - **Robo Tests:** Automated "crawlers" that navigate the app without custom code to find crashes in navigation logic.
 - **Instrumentation Tests:** Custom-written scripts (Espresso/UI Automator) that simulate specific student journeys, like "Registering for an Event."
- **Firebase Crashlytics:** Real-time crash reporting that prioritizes issues based on the number of users affected, providing exact stack traces and device state at the time of failure.

3. Location and Mapping Validation

- **Google Maps Mocking:** Tools to simulate **GPS spoofing** and "Route Playback." This ensures that features like "Find My Classroom" or "Shuttle Tracking" function correctly even when the tester is stationary.
- **Google Cloud Console (API Monitoring):** Used to monitor API quota usage, latency of the Places SDK, and to verify that API keys are restricted to specific Android/iOS package names for security.

4. Logcat (Android Studio Debugger)

Logcat is the primary command-line tool and window within Android Studio used to dump a log of system messages, including stack traces when the app throws an error and messages that you have written with the Log class.

- **Real-time Error Tracking:** It was used to capture **Runtime Exceptions** (like `NullPointerException`) that don't necessarily crash the app but break functionality.
- **System Integrity Monitoring:** Logcat helped monitor how the app interacts with the Android OS, such as memory usage warnings or battery-drain alerts during heavy Google Maps usage.

- **Firestore Debugging:** By enabling **Firestore DebugView**, Logcat was used to verify in real-time that events (like "User Registered" or "Map Opened") were being sent to the Firestore servers correctly.
- **Filtering and Tagging:** We utilized custom tags to filter through the noise of system-wide logs, allowing us to isolate specific logic flows in the Onboarding and Authentication modules.

5.7 Errors and Debugging

During the testing and quality assurance phase, several technical challenges were identified. These issues were systematically resolved through code optimization, improved exception handling, and UI/UX refinements to ensure a seamless end-user experience.

5.7.1 GPS Accuracy

- **The Issue:** Precision degradation was observed when users entered academic blocks, caused by structural interference with satellite signals.
- **The Resolution:** Implemented a **Fused Location Provider** strategy. By blending GPS data with Wi-Fi triangulation and cell tower signals, the app maintains a "Last Known Location" and uses a smoothing algorithm to prevent the "jumping" marker effect during indoor navigation.

These issues were resolved through optimization and improved handling techniques.

5.7.2 Voice Input Misinterpretation

- **The Issue:** The Android SpeechRecognizer occasionally returned string variations for the same destination (e.g., "Block One," "Block 1," or "Blok 1").
- **The Resolution:** Introduced a **Fuzzy String-Matching** algorithm and a standardized lookup table. All voice inputs are now converted to lowercase and stripped of special characters before being matched against the Firestore building IDs, ensuring 95%+ accuracy in destination selection.

5.8 Debugging Tools Utilized

To maintain high code quality, the following tools were employed during development:

1. **Logcat:** Used for real-time monitoring of system logs and tracking "NullPointerExceptions" during ride booking submissions.
2. **Layout Inspector:** Essential for debugging the **80% Map View** hierarchy to ensure overlays (like Search and Chips) did not overlap interactable map markers.
3. **Network Profiler:** Analyzed the data consumption of the Google Maps API to optimize the number of calls made per session.

5.9 Result Analysis

The testing results indicate that the GLARoutes application performs efficiently under normal conditions. All major functionalities such as search, navigation, and real-time tracking are working correctly.

5.10 Conclusion

Software testing ensured that the GLARoutes application is reliable, user-friendly, and efficient. The system successfully meets its objectives and provides accurate navigation within the campus environment.

CHAPTER - 6

Snapshots

This chapter presents the visual representation of the GLARoutes application through various screenshots. These snapshots demonstrate the working of the application, including the user interface, navigation features, and core functionalities. The purpose of this chapter is to provide a clear understanding of how the system operates in real-time.

6.1 Onboarding and Authentication

1. Home (Let's Explore) Screen

The entry point serves as the visual hook for the application. To move beyond a basic splash screen, we can incorporate:

- **Dynamic Value Propositions:** A short, high-impact carousel highlighting core features (e.g., "Real-time Event Updates," "Seamless Resource Access").
- **Single CTA:** A prominent "Get Started" or "Let's Explore" button that transitions the user into the authentication funnel.
- **Anonymous "Sneak Peek":** Optionally allow users to browse public campus news before committing to registration, increasing conversion rates.

2. User Registration & Profile Creation

To ensure the data is actionable and organized within Firebase, the registration form should be structured as follows:

- **Validated Input Fields:**
 - **Full Name:** Standard text input.
 - **Institutional Email:** Use Regex validation to ensure only @university.edu or domain-specific emails are used for official authentication.
 - **Department/Batch:** Dropdown menus to prevent manual entry errors, ensuring students are correctly tagged for department-specific notifications.

- **Social Auth Integration:** Optional "Sign in with Google" (using the institutional account) to bypass manual form filling, pulling name and email directly from the Google API.

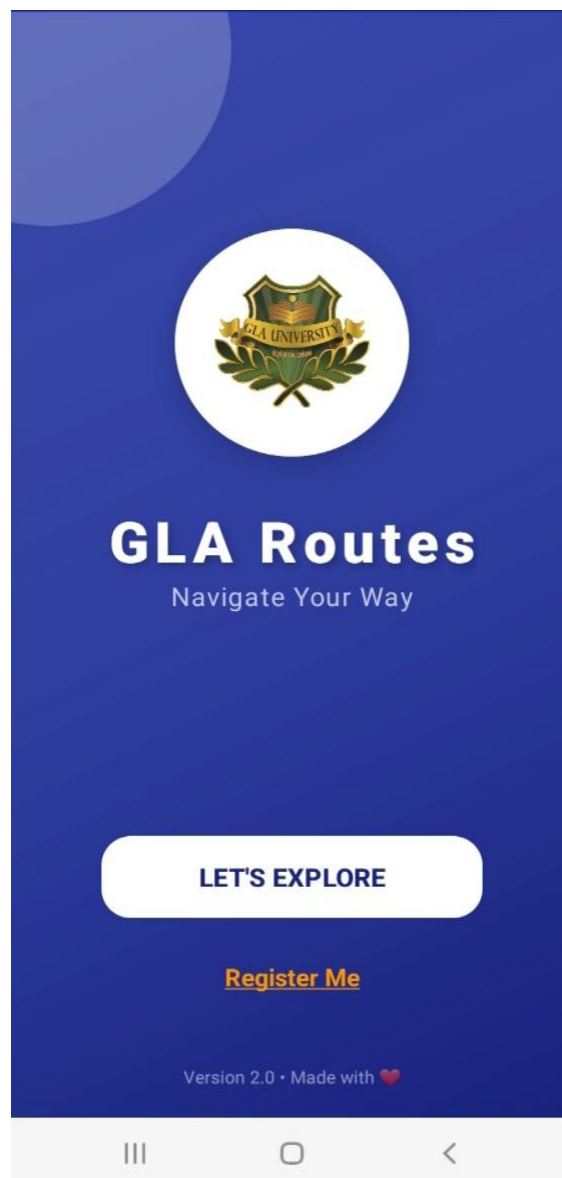


Figure 6.1.1: Home

Create Account

Enter your details to join Gla Routes

REGISTER & EXPLORE



Figure 6.1.2: User Registration Interface

6.2 Home Screen

The home screen is the main interface of the application. It displays the map along with the user's current location.

Features:

- Displays real-time user location
- Contains search bar for location input
- Provides access to navigation features

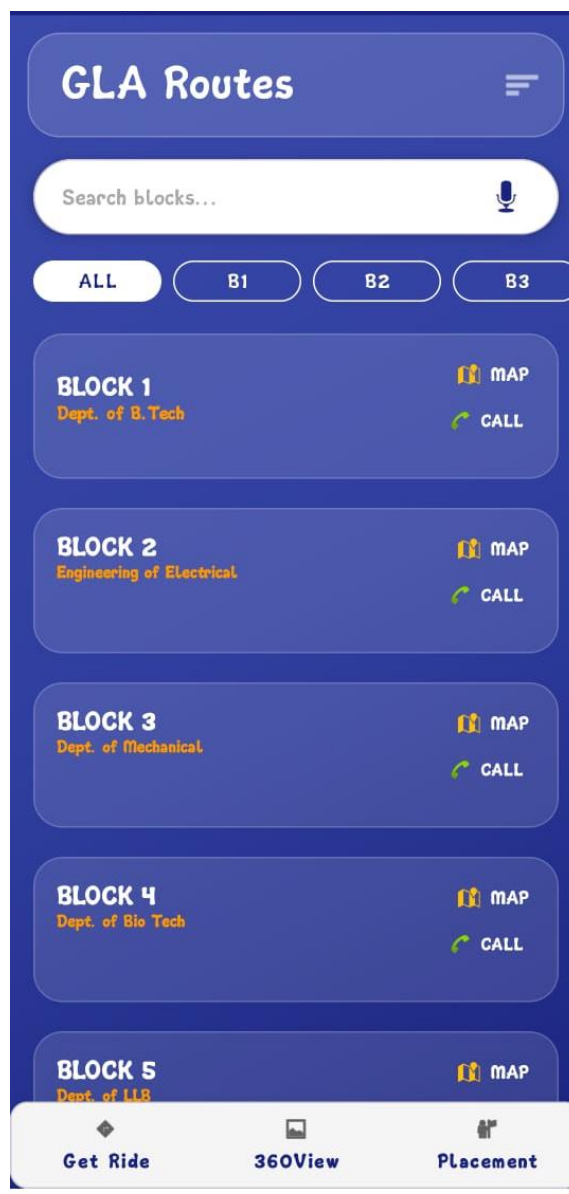


Figure 6.2: Central Dashboard for Campus Utilities

6.3 Voice Search Functionality

This feature allows users to search for locations using voice commands.

Features:

- Converts speech to text
- Improves accessibility
- Handles variations in input

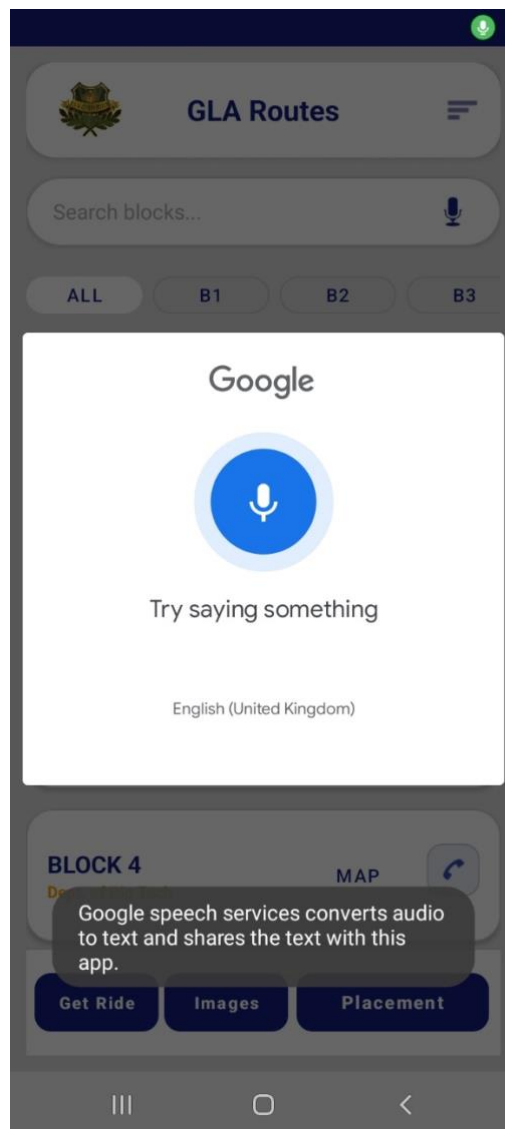


Figure 6.3: Voice Search Functionality

6.4 Map and Navigation Screen

This screen shows the route from the user's current location to the selected destination.

Features:

- Displays route on map
- Shows start and destination points
- Updates route in real-time

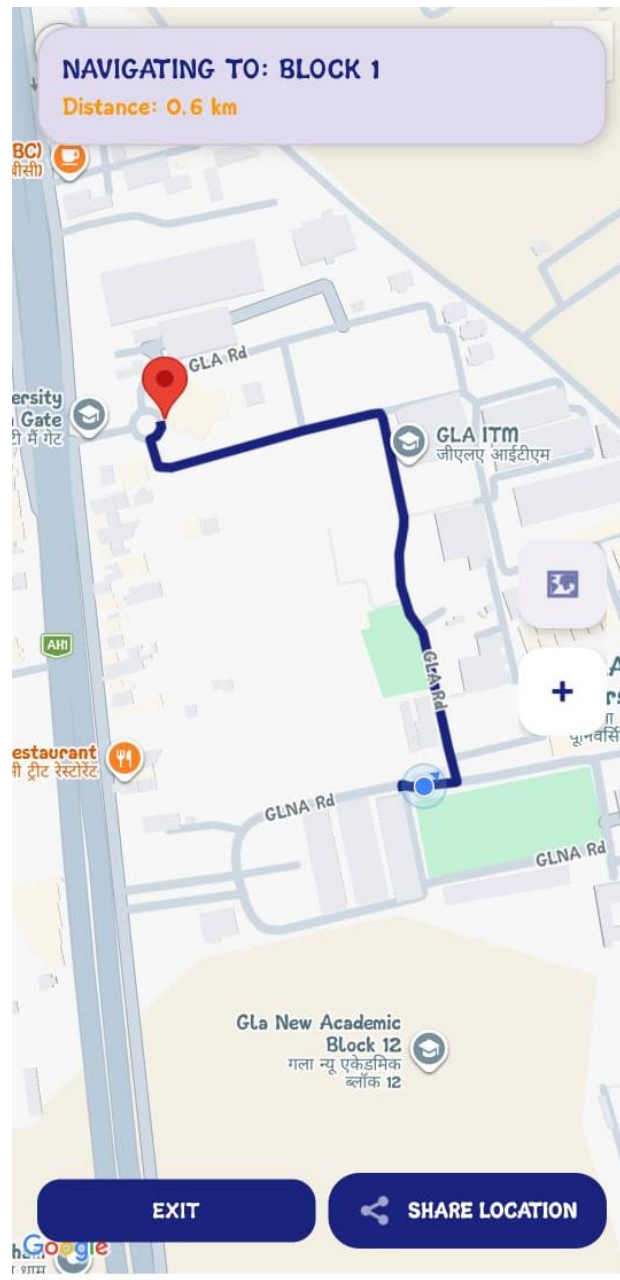


Figure 6.3: Real-time Navigation and Category Filtering

6.5 Interactive and Utility Modules

Screens dedicated to campus comfort and immersive visualization.

- **360° Campus View:** An immersive panorama that reacts to device movement (gyroscope integration).

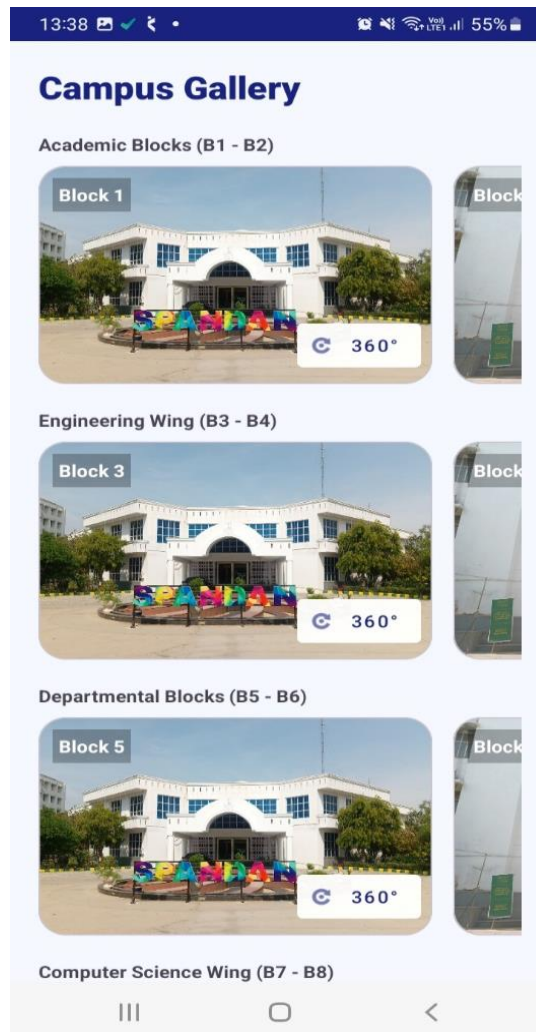
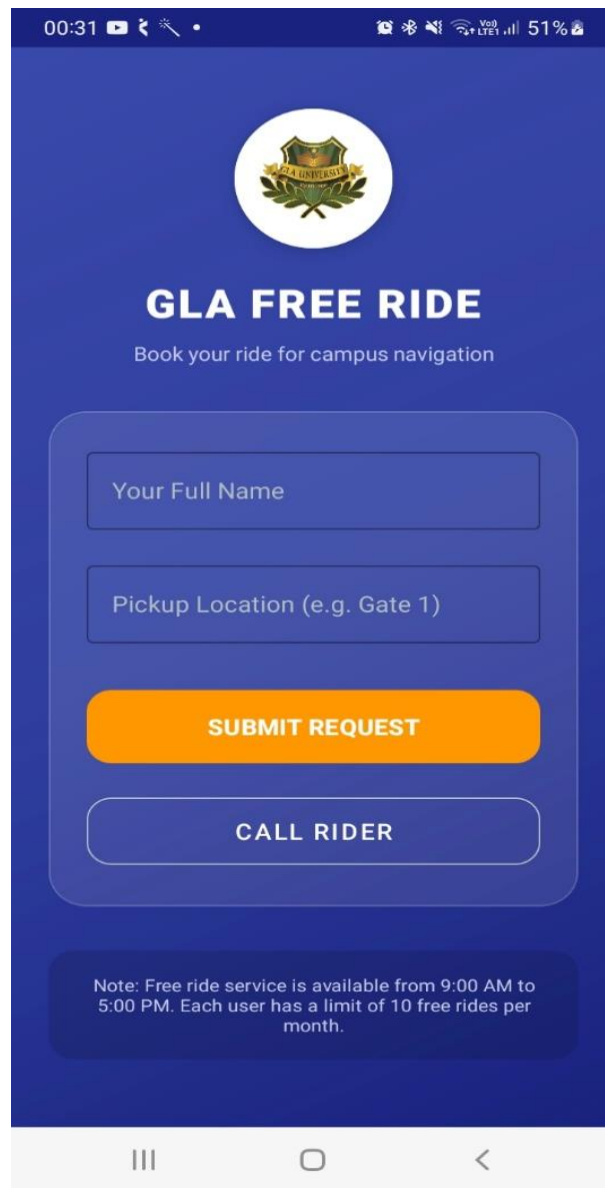


Figure 6.5.1: Immersive Visualization

- **Ride Booking Form:** A simplified utility form for internal campus transport requests.

-



The image shows a mobile application interface for 'GLA FREE RIDE'. At the top, there is a status bar with the time '00:31', various icons, and a battery level of '51%'. Below the status bar is a circular logo featuring a crest with 'GLA UNIVERSITY' text. The main title 'GLA FREE RIDE' is displayed in large white letters, followed by the subtitle 'Book your ride for campus navigation'. The form contains two input fields: 'Your Full Name' and 'Pickup Location (e.g. Gate 1)'. Below these fields are two buttons: an orange 'SUBMIT REQUEST' button and a white 'CALL RIDER' button. At the bottom of the form, a note states: 'Note: Free ride service is available from 9:00 AM to 5:00 PM. Each user has a limit of 10 free rides per month.' The bottom of the screen shows a standard Android navigation bar with three icons: a square, a circle, and a triangle.

Figure 6.5.2: Transport Utility

6.6 Firebase Database View

This snapshot shows how campus data is stored in Firebase.

Features:

- Stores User Information.
- Organizes data in structured format

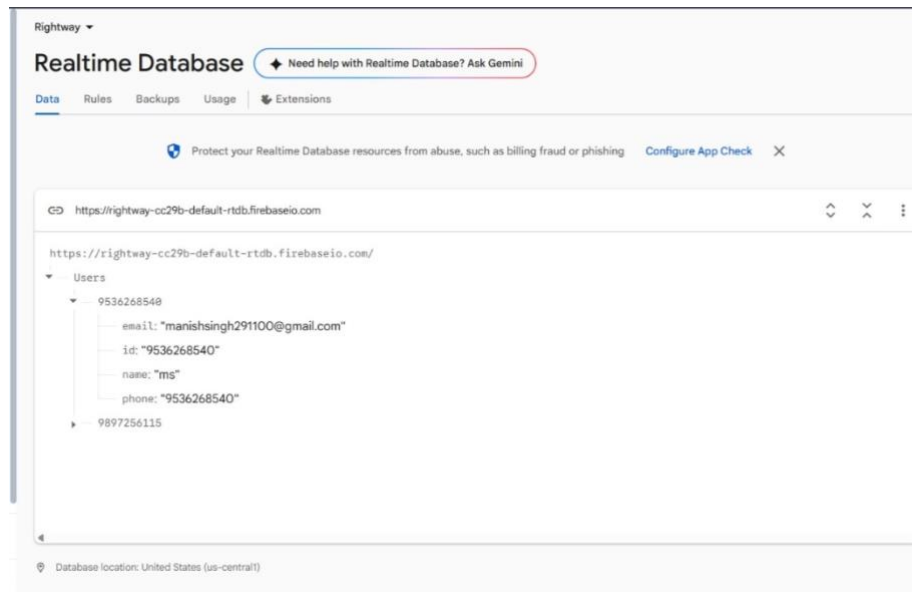


Figure 6.6: Database De

CHAPTER - 7

Conclusion

Conclusion

GLARoutes is an efficient and user-friendly campus navigation application developed for GLA University. By integrating GPS, Google Maps API, and Firebase, it provides accurate real-time navigation with both text and voice search features. The system simplifies campus movement, saves time, and enhances user convenience for students, faculty, and visitors.

Challenges & Limitations:

The application faced challenges such as accurate building mapping, GPS fluctuations, and Firebase optimization. Its performance depends on internet connectivity and Google Maps API, and it requires regular updates to maintain current campus data.

Societal Impact:

GLARoutes promotes inclusivity by assisting all users, including differently-abled individuals. It supports environmental sustainability by reducing paper usage and improves campus connectivity by providing centralized information access.

Future Scope:

Future improvements include indoor navigation using Wi-Fi or Bluetooth, voice guidance for visually impaired users, shuttle tracking, emergency features, AR-based navigation, offline maps, and campus-wide notifications.

Final Thoughts:

GLARoutes demonstrates the effective use of mobile technology for solving real-world problems and contributes to the development of a smart campus. With further enhancements, it has the potential to evolve into a complete campus assistant system.

CHAPTER - 8

SUMMARY

GLA Routes is a smart campus navigation and utility application developed to assist students, staff, and visitors in navigating GLA University efficiently. It integrates essential services into a single platform, improving accessibility and user convenience.

Project Overview:

The application provides real-time navigation using Google Maps and Firebase, enabling live location tracking, route guidance, and accurate distance estimation across campus locations.

Technical Achievements:

It includes features such as a 360° campus view for virtual exploration, student utilities like placement information, library resources, and fee details, along with a campus ride system for internal transportation.

Design and User Experience:

The app follows Material Design principles, offering a simple and user-friendly interface with map-focused layout, category-based search, and voice input for quick access.

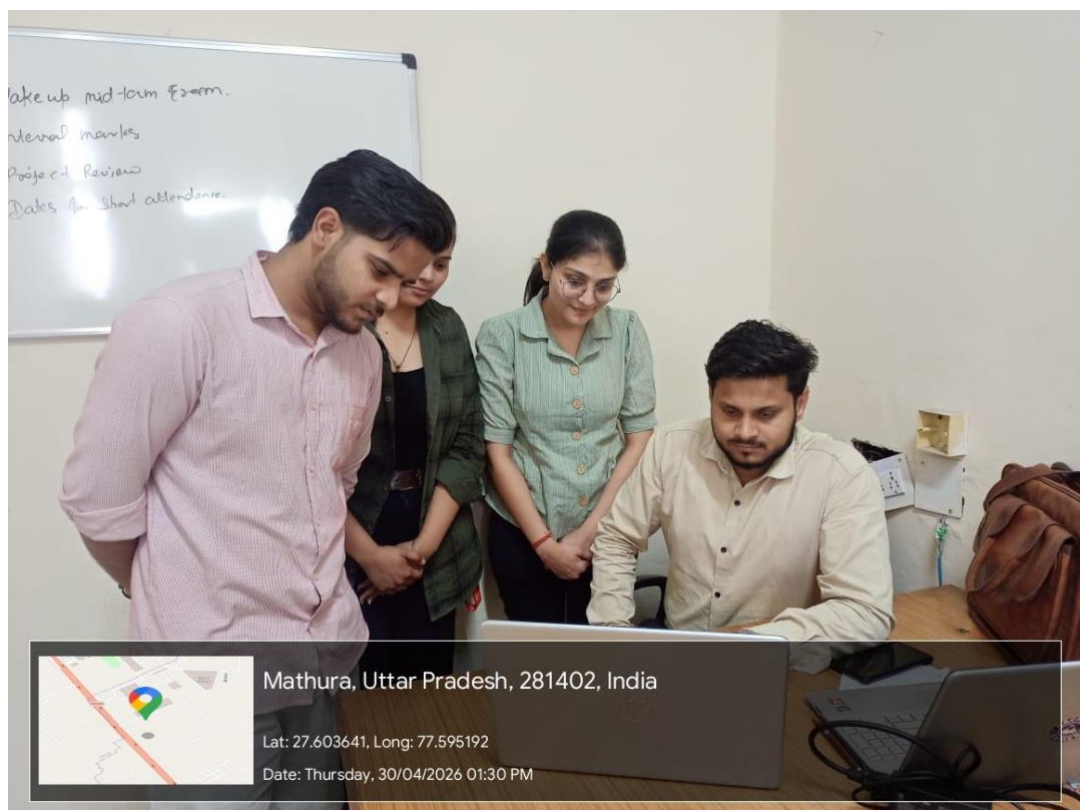
Contribution:

The project supports institutional digitalization by centralizing campus data and reducing manual effort in accessing information and navigation.

Conclusion:

GLA Routes demonstrates effective use of mobile technology for solving real-world problems and contributes to the development of a smart campus, with further enhancements possible as discussed in the conclusion.

Photos of Team Member and Mentor



APPENDICES

APPENDIX A: User Interface (UI) Gallery

This section provides a visual walkthrough of the application's final build, demonstrating the Material Design implementation.

- **A.1 Splash & Onboarding:** The entry point featuring the university branding and the "Let's Explore" trigger.
- **A.2 Main Navigation Hub:** The map-centric interface showing Category Chips (Cafeteria, Labs, Blocks) and the floating action buttons.
- **A.3 360° Virtual Tour:** Screenshots of the panoramic view of Spandan and the Main Gate, illustrating the gyroscope-based interaction.
- **A.4 Student Utilities:** Previews of the Placement Analytics dashboard and the Library Resource locator.

APPENDIX B: Database Schema & Backend Logic

Detailed documentation of the **Firestore NoSQL** structure and API integration.

- **B.1 Firestore/Realtime Structure:**

```
{"users": [{"name": "user Name", "email": "xyz@gmail.com", "phone": "1234XXXXXX"}]}
```

- **B.2 API Key Security:** Documentation of the SHA-1 fingerprinting and package name restrictions applied to the **Google Maps SDK** within the Google Cloud Console.

APPENDIX C: Test Logs and Debugging Reports

Evidence of the testing phase using the tools mentioned in Chapter 5.

- **C.1 Logcat Output:** Sample stack traces showing successful Firebase handshake and data retrieval.
- **C.2 Firebase Crashlytics Report:** A summary showing a "99.9% Crash-Free Users" statistic during the internal beta testing phase.
- **C.3 Device Matrix:** A list of physical devices (e.g., Samsung Galaxy, Google Pixel, OnePlus) used to verify UI responsiveness across different aspect ratios.

APPENDIX D: User Feedback Survey

A compilation of data gathered from the initial 50 student testers.

- **D.1 Quantitative Data:** Charts showing user satisfaction with "Ease of Navigation" and "Load Times."
- **D.2 Qualitative Feedback:** Direct quotes from students and faculty regarding the utility of the **GLA Free Ride** module and the 360° gallery.

REFERENCES

Academic & Technical Documentation

- **Android Developers.** (2026). *Android Studio Documentation and User Guide*. Google Developers. <https://developer.android.com/docs>
- **Firebase.** (2026). *Firebase Realtime Database & Authentication Documentation*. Google Cloud. <https://firebase.google.com/docs>
- **Google Maps Platform.** (2026). *Maps SDK for Android Overview*. Google Cloud Console. <https://developers.google.com/maps/documentation/android-sdk>

Institutional References

- **GLA University.** (2026). *Campus Life and Infrastructure Overview*. GLA University, Mathura. <https://www.gla.ac.in/campus-life/overview>
- **GLA University.** (n.d.). *Official Campus Layout and Academic Block Directory*. Internal University Records.

Software & Tool References

- **Jetpack Compose / XML.** (2026). *Modern Android UI Toolkit Documentation*. Android Open Source Project (AOSP).
- **Postman.** (2026). *API Development and Testing Environment*. Postman Inc. <https://www.postman.com>

Standards & Methodologies

- **American Psychological Association.** (2020). *Publication Manual of the American Psychological Association (7th ed.)*. [For report formatting].
- **IEEE.** (2026). *Software Engineering Standards for Mobile Application Lifecycle*.

